

AI-Powered Resume Screening System

Final Viva Preparation Guide - Hinglish Version

Project: AI in Recruitment Process: A Study on the Use of AI for Resume Analysis

Team: Rohan Garg, Punit Sukhani, Saksham | Guide: Mrs. Gull Kaur



Delhi Technological University

1. Project ka Short Summary

Ye project ek AI-powered resume screening prototype hai. System ek Job Description (JD) aur multiple resumes leta hai, sab files se text extract karta hai, skills aur years of experience detect karta hai, weighted score calculate karta hai, score ka explanation generate karta hai, aur final dashboard par candidates ko rank karke dikhata hai.

Viva line: Hamara system HR ko replace nahi karta. Ye ek explainable first-pass filter hai jo initial shortlisting fast, transparent aur low-cost banata hai.

2. Problem kya solve kar rahe hain?

Recruiters ko ek job opening ke liye bahut saare resumes milte hain. Manual screening slow hoti hai, inconsistent ho sakti hai, aur bias ka chance hota hai. Traditional ATS mostly keyword matching karta hai, isliye agar candidate ne same wording use nahi ki, to relevant candidate miss ho sakta hai. LLM models semantic understanding me better hote hain, but costly, slow, memory-heavy aur less explainable hote hain. Isliye humne lightweight Regex + TF-IDF + XAI based prototype banaya.

- **Manual issue:** time-consuming aur inconsistent.
- **Keyword issue:** synonyms miss ho sakte hain, keyword stuffing possible hai.
- **LLM issue:** cost, latency, memory aur black-box explainability.
- **Our solution:** fast, transparent, auditable first-pass screening.

3. Input se Output tak Flow

Input: recruiter login, JD text, resumes in PDF/DOCX/TXT/PNG/JPG, aur scoring weights. **Output:** ranked candidates, final score, matched/missing skills, soft skills, YoE status, bonus skills, charts, aur XAI explanation.

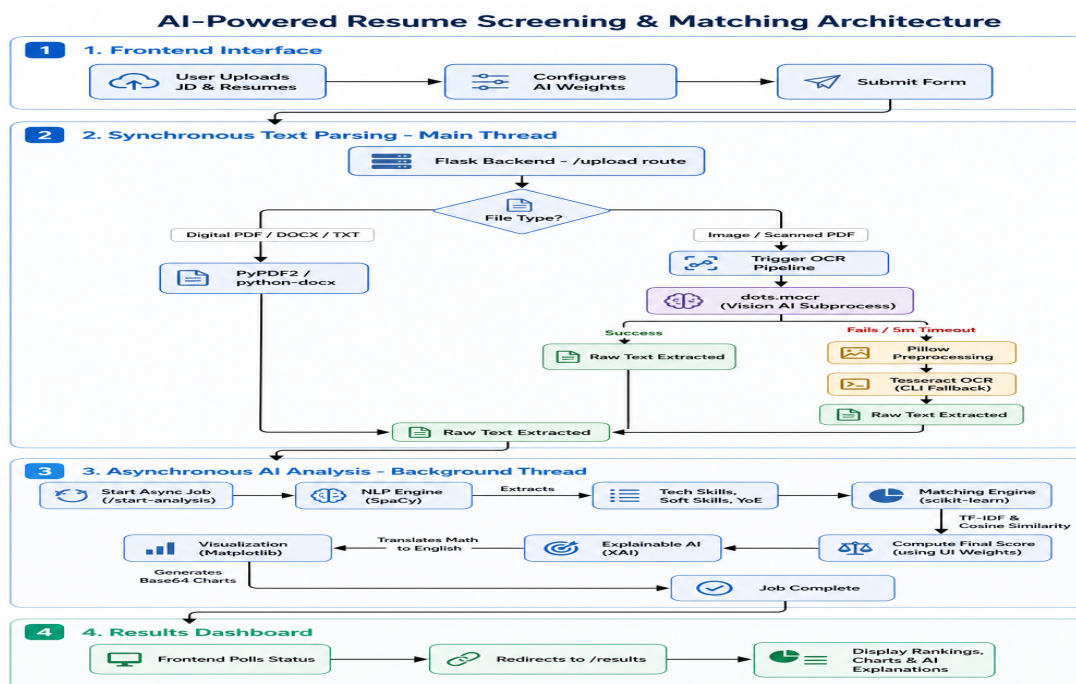


Figure: Complete flow - frontend upload se parser, OCR, NLP, matching, XAI aur result dashboard tak.

Step-wise system flow

- 1 Recruiter login/register karta hai.
- 2 Upload screen par JD paste karta hai aur resumes upload karta hai.
- 3 Recruiter weights set karta hai: technical skills, experience, context, soft skills, bonus cap.
- 4 Backend files validate karta hai aur data UUID key ke saath store karta hai.
- 5 Analyze page background job start karta hai aur status poll karta hai.

- 6 Parser text extract karta hai; scanned/image files OCR pipeline me jaati hain.
- 7 NLP module skills, soft skills aur years of experience extract karta hai.
- 8 Matching engine component scores aur final weighted score calculate karta hai.
- 9 XAI module score ko plain-English explanation me convert karta hai.
- 10 Dashboard ranked results, charts, tags aur explanation panels dikhata hai.

4. Components aur Responsibilities

Layer	Technology	Why it is used
Backend	Python, Flask	Lightweight REST-style web backend; easy to split parsing, NLP, matching, XAI and dashboard modules.
Frontend	HTML5, Bootstrap 5, Jinja2	Responsive pages with server-rendered score breakdowns and explanation panels.
Database/Auth	SQLite, SQLAlchemy, Flask-Login, Authlib	Simple local persistence, login protection, optional Google OAuth SSO.
Parsing	PyPDF2, python-docx, Pillow, pdf2image, Tesseract	Extracts text from digital files and OCRs scanned/image resumes.
NLP	SpaCy + Regex	Tokenization, entities, strict skill extraction, and YoE extraction.
Matching	scikit-learn TF-IDF + Cosine Similarity	Fast statistical context matching without heavy transformer models.
XAI/Charts	Custom Explainer, Matplotlib, Chart.js	Human-readable explanations and visual score dashboards.

Important source modules

- **app.py:** Flask app create karta hai.
- **models.py:** User model for auth.
- **resume_parser/parser.py:** PDF/DOCX/TXT/image parsing aur OCR fallback.
- **nlp_processing/processor.py:** SpaCy, skills, soft skills, YoE extraction.
- **feature_extraction/extractor.py:** TF-IDF vectorizer.
- **matching_engine/matcher.py:** weighted scoring formula.
- **explainable_ai/explainer.py:** human-readable explanation.
- **visualization/charts.py:** score and skill charts as base64 images.
- **web_app/routes.py:** upload, analyze, status polling, results.

5. Parsing aur OCR Pipeline

AI processing se pehle har resume ko plain text me convert karna zaroori hai. Parser pehle digital extraction try karta hai. Agar PDF se 50 characters se kam text aata hai, to system assume karta hai ki file scanned ya parser-resistant hai, aur OCR fallback trigger karta hai. Isse format bias reduce hota hai.

Format	Method	Explanation
.txt	Direct read	Seedha text read hota hai.
.docx	python-docx	Word XML paragraphs se text nikalta hai.
.pdf	PyPDF2	Agar text weak hai, PDF images me convert karke OCR.
.png/.jpg	Pillow + OCR	Image preprocessing ke baad OCR.

OCR preprocessing me transparency flattening, grayscale conversion, contrast boost, sharpening, binarization, aur Tesseract CLI extraction hota hai. Direct Tesseract CLI use kiya gaya hai because pytesseract wrapper me decode issues aa sakte hain.

Algorithm: OCR-aware parsing

```
FUNCTION ParseDocument(file, extension):
  IF .txt: read text directly
  IF .docx: extract paragraphs using python-docx
  IF .pdf:
    text = PyPDF2_extract(file)
    IF length(text) < 50:
      convert PDF pages to images
      return OCR_Pipeline(images)
  IF image file:
    return OCR_Pipeline(file)

FUNCTION OCR_Pipeline(image):
  flatten transparency
  grayscale -> contrast -> sharpen -> binarize
  return Tesseract text
```

6. NLP aur Regex Skill Extraction

NLP layer SpaCy ka en_core_web_sm model use karta hai tokenization, lemmatization aur entities ke liye. But technical skills generic NER se nahi nikalte; curated skills database ke against strict regex word-boundary matching use hoti hai. Reason: precision. Example: skill 'C' ko 'communication' ke andar match nahi hona chahiye.

```
pattern = r'\b' + re.escape(skill) + r'\b'
```

YoE extraction three-step cascade follow karta hai: direct phrase ('5 years experience'), reversed phrase ('Experience: 5 years'), aur date ranges ('June 2024 - August 2025'). Invalid values filter hote hain, and education-duration inflation avoid kiya jata hai.

7. TF-IDF Context Matching

TF-IDF resume aur JD ko vectors me convert karta hai. Project me max_features=5000, stop_words='english', aur ngram_range=(1,2) use hota hai. Bigrams phrases jaise machine learning capture karte hain. Cosine similarity check karti hai ki resume aur JD same direction/topics me align hain ya nahi.

Formula: Context Score = $\text{dot}(A,B) / (||A|| \times ||B||)$. Yahan A resume vector hai aur B JD vector hai.

8. Matching Formula

Score	Calculation	Viva explanation
Tech	matched tech / required tech	Hard technical fit.
Soft	matched soft / required soft	Communication/teamwork etc.
YoE	$\min(\text{resume_y} / \text{jd_y}, 1.0)$	Over-qualified score cap.
Context	TF-IDF cosine	Broader textual similarity.
Bonus	capped extra skills	Adjacent skills reward, but limited.

Final Score =
Tech*Wtech + Soft*Wsoft + YoE*WyoE + Context*Wcontext + Bonus

Default:
Tech 60%, YoE 15%, Context 15%, Soft 10%, Bonus cap 5%

9. Explainable AI (XAI)

XAI module score dictionary ko directly read karke explanation banata hai. Isliye ye black-box explanation nahi hai. It lists matched skills, missing skills, extra skills, experience status, score contribution and match band.

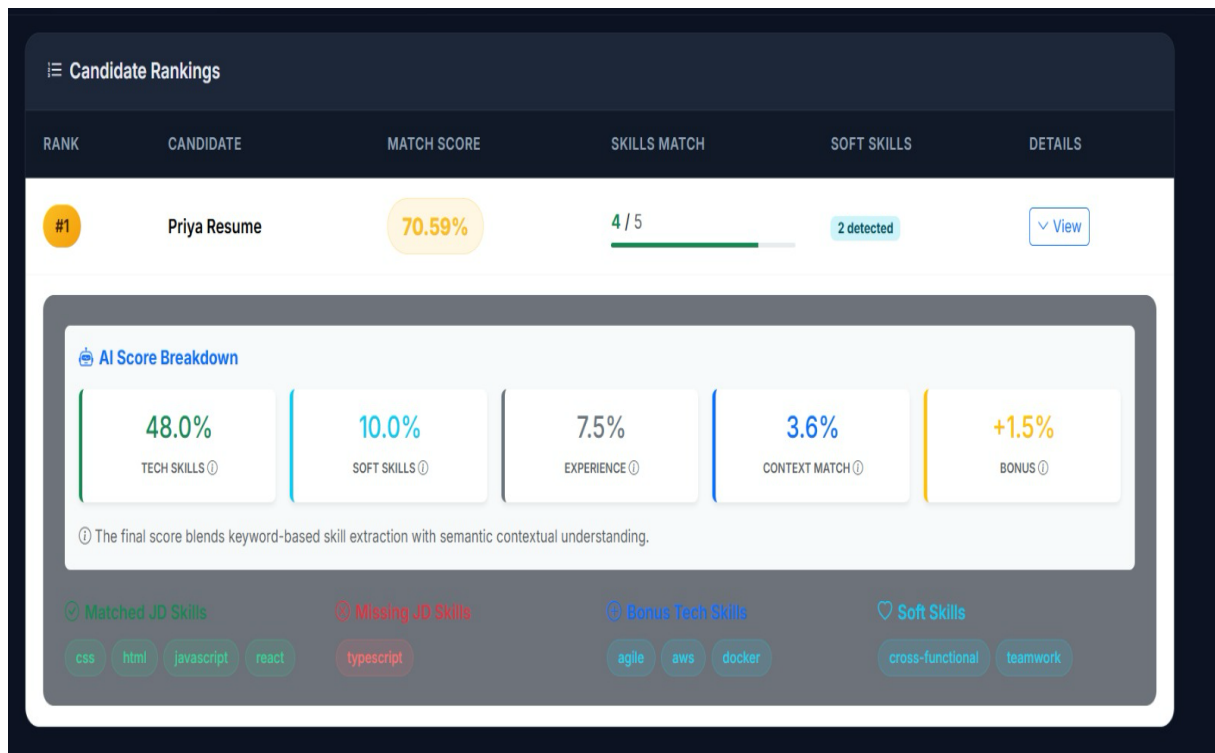


Figure: XAI breakdown - score components, matched skills, missing skills, bonus skills.

Match bands: Strong $\geq 75\%$, Moderate $\geq 50\%$, Partial $\geq 30\%$, Weak $< 30\%$.

10. Frontend, Sliders aur Async Backend

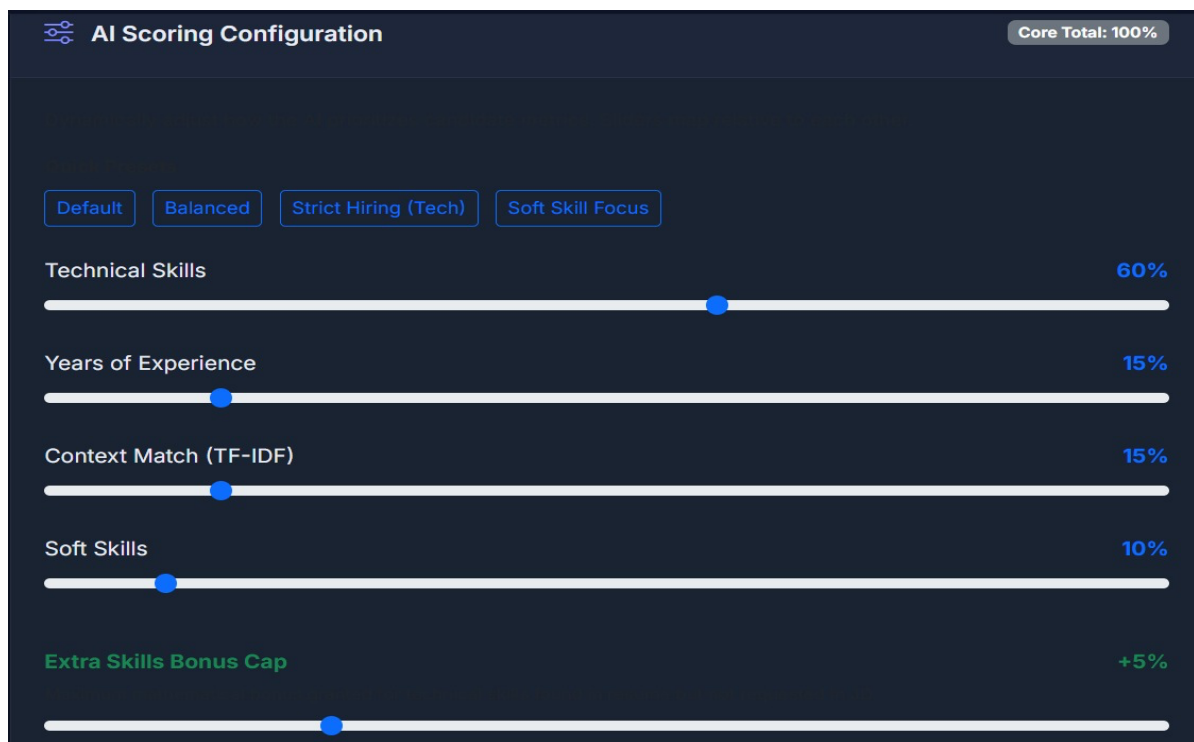


Figure: Recruiter-controlled scoring weights. Bonus cap separately handled होता है.

Upload dashboard me 5 sliders hain: technical skills, YoE, context, soft skills, bonus cap. First 4 sliders ka total 100% maintain होता है using `normalizeSliders()`. Bonus alag rakha gaya है because it is additive, not part of core normalized score.

Backend async architecture use karta है. Upload ke baad analyze page `/start-analysis` hit karta है, background daemon thread analysis run karta है, aur frontend `/analysis-status/` ko poll karta है. Isse browser freeze nahi होता.

```

POST /upload -> validate and store data
GET /analyze -> progress page
POST /start-analysis -> background thread
GET /analysis-status/<job_id> -> progress polling
GET /results -> ranked dashboard

```

11. Benchmark Results

Benchmark	What was tested	Key result / viva point
1. Format Bias & OCR	TXT vs PDF vs PNG extraction and scoring.	PNG through OCR scored 90.5%, same as TXT. PDF scored 90.0% due to spacing loss.
2. Keyword Stuffing	Baseline, verbose/diluted resume, and keyword stuffer.	Regex kept hard-skill score stable; TF-IDF penalized dilution and contained stuffing.
3. XAI & Boundaries	Over-qualified YoE and extra skills bonus.	YoE capped at 100%; extra bonus +2.0% under 5.0% cap; explanation remained auditable.
4. A vs B vs C	Current Regex+TF-IDF vs Sentence-BERT vs future hybrid.	LLM understood semantics better; current model stayed strict and explainable.
5. Efficiency	100-resume stress test.	Approach A: 0.0062 sec, 0.12 MB; much faster/lighter than LLM alternatives.

Approach A/B/C ka meaning

- **Approach A:** Current app - Regex + TF-IDF. Fast, explainable, CPU-friendly.
- **Approach C:** Pure LLM/Sentence-BERT. Semantic understanding better, but costly and less bounded.
- **Approach B:** Future hybrid - regex hard-skill shield + LLM context. Better future direction but heavier.

12. Strengths, Limitations, Future Work

Strengths	Limitations	Future work
Fast, low-cost, OCR fallback, XAI, dynamic weights, keyword-stuffing resistance.	Regex synonyms miss kar sakta hai; TF-IDF deep meaning nahi samajhta; skills DB update karni padti hai; OCR scan quality par depend karta hai.	Sentence-BERT hybrid, dynamic skill graph, fairness audit, recruiter feedback, production DB, larger real dataset.

13. Viva Demo Script

- 1 App run karo: python app.py, then browser me http://127.0.0.1:5001.
- 2 Login/register dikhao.
- 3 Upload screen par JD paste karo and resumes upload karo.
- 4 Weight sliders explain karo: Tech 60, YoE 15, Context 15, Soft 10, Bonus cap 5.
- 5 Submit karke analyze/progress page dikhao.
- 6 Results dashboard me ranking, score breakdown, matched/missing skills explain karo.
- 7 XAI panel open karke batao ki score black-box nahi hai.
- 8 End me honest limitation bolo: synonyms ke liye future hybrid Sentence-BERT useful hoga.

14. Viva Q&A; Quick Sheet

Question	Answer
Regex kyun?	Strict word boundaries precision dete hain and false positives avoid karte hain.
TF-IDF kyun?	Fast, interpretable, CPU-friendly lexical context matching.
LLM kyun nahi?	LLM semantic better hai but cost/latency/memory and explainability issue hai.
XAI kya hai?	Score components ko human-readable explanation me convert karna.
OCR kyun?	Scanned/image resumes ko process karke format bias reduce karna.
Main limitation?	Regex implicit skills/synonyms miss kar sakta hai.
Future scope?	Hybrid Sentence-BERT context, dynamic skill ontology, fairness audit.

15. Ma'am ko Algorithm kaise explain karna hai

Agar ma'am pooche ki input se output tak algorithm kya hai, to project ko deterministic pipeline ki tarah explain karna. Important point: har candidate same stages se pass hota hai, isliye ranking consistent hoti hai.

```
INPUT:
Job Description J
Resume batch R = {r1, r2, ..., rn}
Recruiter weights W
Bonus cap Bmax

PROCESS:
1. JD ka text extract karo.
2. JD se required skills, soft skills, YoE nikalo.
3. Har resume ke liye:
  a. File type detect karo.
  b. Digital text extraction try karo.
  c. Text weak ho to OCR fallback chalao.
  d. Text normalize karo.
  e. Candidate skills, soft skills, YoE extract karo.
  f. Tech, soft, YoE, context scores calculate karo.
  g. Extra skills bonus cap ke saath calculate karo.
  h. Final weighted score banao.
  i. XAI explanation generate karo.
4. Candidates ko final score ke according sort karo.

OUTPUT:
Ranked list + score breakdown + explanation + charts.
```

16. Backend Routes ka Flow

Route / Page	Kya hota hai	Viva me kya bolna hai
/upload	JD, resumes and weights receive karta hai.	Yahi se AI pipeline input leti hai.
/analyze	Progress page dikhata hai.	User ko freeze feel nahi hota.
/start-analysis	Background thread start karta hai.	Heavy analysis request thread ko block nahi karti.
/analysis-status/	Current status return karta hai.	Frontend 2 sec interval me poll karta hai.
/results	Ranked output render karta hai.	Scores, charts, skills and explanations show hote hain.

17. Internal Data Objects yaad rakhna

- **NLP output:** tokens, lemmas, entities, technical_skills, soft_skills, years_of_experience.
- **Matcher output:** final_score, text_similarity, tech_score, soft_score, yoe_score, contributions, bonus.

- **Explainer output:** matched skills, missing skills, extra skills, detected soft skills, final explanation.
- **Job status:** queued, loading_models, processing_jd, processing_resumes, computing_scores, generating_explanations, creating_charts, done.

18. Design Choices ka Reason

Choice	Reason	Trade-off
Flask	Lightweight and modular prototype ke liye easy.	Django jitna built-in structure nahi.
SQLite	Local DB, no server needed.	High-concurrency production ke liye ideal nahi.
Regex skills	Precise and auditable hard-skill matching.	Synonyms miss ho sakte hain.
TF-IDF	Fast, interpretable, CPU-friendly context score.	BERT jaisa semantic nahi.
OCR fallback	Scanned/image resumes process karne ke liye.	Image quality par depend karta hai.
Async thread	Browser/request block nahi hota.	Production me Celery/Redis better hoga.

19. Last-Minute Revision Checklist

- One-line pitch yaad rakho: explainable first-pass resume screening system.
- Flow draw kar pao: Upload -> Parse/OCR -> NLP -> TF-IDF -> Score -> XAI -> Dashboard.
- Default weights yaad rakho: Tech 60, YoE 15, Context 15, Soft 10, Bonus 5.
- Benchmark numbers: TXT/PNG 90.5%, PDF 90.0%, Approach A 0.0062 sec and 0.12 MB.
- Limitations confidently bolo: regex synonyms miss karta hai, TF-IDF deep semantics nahi samajhta, OCR scan quality par depend karta hai.
- Future scope bolo: Sentence-BERT hybrid, fairness audit, dynamic skill ontology, recruiter feedback loop.

20. Final Closing Line

Hamara project ek practical, explainable, low-cost first-pass resume screening system hai. Itna heavy nahi ki LLM jaisa expensive ho, aur itna simple bhi nahi ki sirf keyword counter ban jaye. Iska main value speed, transparency, OCR resilience, dynamic scoring, aur recruiter-friendly explanation hai.